Resource management and autoscaling

Requests, limits and QoS, CPU throttling and OOM, quotas and priority, eviction, HPA, VPA, node autoscaling with Karpenter and KEDA, and disruption planning.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Requests, limits and QoS classes (12 min)
2. CPU and memory behaviour: throttling and out-of-memory kills (12 min)
3. LimitRange, ResourceQuota and priority classes (12 min)
4. Node allocatable, pressure and eviction (12 min)
5. Horizontal pod autoscaling (13 min)
6. Vertical scaling and right-sizing (12 min)
7. Cluster autoscaling, Karpenter and KEDA (12 min)
8. Disruption budgets, topology spread and capacity planning (13 min)
Cheat sheet

Lesson 1. Requests, limits and QoS classes

12 minutes. Know what requests and limits mean to the scheduler and the kernel, and how they define the Guaranteed, Burstable and BestEffort QoS classes.

Every container can declare how much CPU and memory it needs and how much it may use. Those two numbers are read by different components for different purposes, and getting them wrong is the most common cause of both wasted money and unexpected restarts.

What you will learn

- The difference between a request and a limit
- How the scheduler uses requests and the kernel uses limits
- CPU units and memory units
- QoS classes and their eviction order
- How to set sensible values

Request and limit

A request is the amount the container is guaranteed and what the scheduler reserves when it places the pod. A limit is the most it may use. CPU is measured in cores, with `500m` meaning half a core. Memory is in bytes, written as `256Mi` or `1Gi`. Requests above what any node can offer make the pod unschedulable.

Who enforces what

The scheduler sums requests of pods on a node and compares them to the node allocatable resources. It never looks at actual usage. At runtime the kernel enforces limits through cgroups. CPU is compressible: a container at its CPU limit is throttled, not killed. Memory is not: a container that exceeds its memory limit is killed by the out-of-memory killer and restarted.

QoS classes

The kubelet assigns every pod a quality of service class from its requests and limits. Guaranteed means every container has both CPU and memory, with requests equal to limits. Burstable means at least one request or limit is set but the pod is not Guaranteed. BestEffort means nothing is set. Under node memory pressure, the kubelet evicts BestEffort pods first, then Burstable pods using more than their requests, and Guaranteed pods last.

```
kubectl get pod web-1 -o jsonpath='{.status.qosClass}{"\n"}'
kubectl describe node worker-1 | sed -n '/Allocated resources/,/Events/p'
```

Setting values

Start from measurement. Look at real usage over a representative period, and set the request near typical or percentile usage and the memory limit with headroom above the peak. Many teams set memory request equal to limit for predictability and leave CPU limits unset or generous to avoid throttling. Latency sensitive and critical pods suit Guaranteed.

```
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    memory: 512Mi
```

A quick calculation

```
node_alloc_cpu = 3.8
req = {"web": 0.25, "api": 0.5, "cache": 1.0}
replicas = {"web": 4, "api": 3, "cache": 1}
used = sum(req[k] * replicas[k] for k in req)
print("requested:", used, "of", node_alloc_cpu, "free:", round(node_alloc_cpu - used, 2))
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Requests, limits and QoS classes

Part	What it does	What goes wrong
Container spec	Requests and limits.	Missing values.
Scheduler	Reserves requests.	Overcommitted.
cgroups	Enforce limits.	Throttle or OOM.
QoS class	Sets eviction order.	BestEffort.

Flow: Container spec to Scheduler / Container spec to cgroups / cgroups to QoS class

Incident: OOMKilled with plenty of node memory

Incident: *A pod restarted every few hours while the node showed large free memory.*

```
$ kubectl describe pod api-5c7d | grep -A4 "Last State"
    Last State:     Terminated
      Reason:       OOMKilled
      Exit Code:    137
$ kubectl get pod api-5c7d -o jsonpath='{.spec.containers[0].resources.limits.memory}'
256Mi
(this is a simulated illustration, not a real incident)
```

The memory limit applies per container, not per node. The application grew beyond 256 MiB during a batch job and the kernel killed it at its cgroup limit, regardless of free node memory. Raising the limit to match measured peaks plus headroom stopped the restarts, and a graph of container memory against its limit was added. Exit code 137 with OOMKilled is the signature.

Common mistakes

- Setting no requests at all
- Setting limits far above requests everywhere
- Setting memory limits below real peak usage
- Expecting CPU limits to kill containers
- Ignoring QoS when a node is under pressure

Check yourself

What does the scheduler use to place a pod?

Answer: Requests

What happens when a container exceeds its CPU limit?

Answer: It is throttled

What happens when it exceeds its memory limit?

Answer: It is OOM killed and restarted

Which pods are evicted first under memory pressure?

Answer: BestEffort

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. CPU and memory behaviour: throttling and out-of-memory kills

12 minutes. Understand CFS quota throttling, cpu shares, memory working sets and OOM scoring, so performance symptoms can be tied to resource settings.

Resource settings turn into kernel settings, and the kernel reacts in specific ways. Learning those reactions lets you tell throttling from a slow dependency and an OOM kill from an application crash.

What you will learn

- How CPU requests become shares and limits become quota
- What throttling looks like and how to measure it
- Working set memory and what counts toward the limit
- The OOM killer, oom_score_adj and exit code 137
- Practical tuning

CPU shares and quota

A CPU request sets cgroup cpu shares, a relative weight used when the node is busy. A CPU limit sets a CFS quota, for example 50 ms of CPU time in each 100 ms period for a limit of 0.5 cores. A multi-threaded application can use its whole quota in a few milliseconds and then be paused for the rest of the period, which shows up as latency spikes even when average CPU looks low.

Measuring throttling

The metric `container_cpu_cfs_throttled_periods_total` against `container_cpu_cfs_periods_total` gives the throttled share. A pod that is throttled in a large share of periods needs a higher limit, or no limit, or fewer threads. Many teams drop CPU limits for latency sensitive workloads and rely on requests for fairness, while keeping namespace level quotas.

```
kubectl top pod -n shop
kubectl exec api-1 -- cat /sys/fs/cgroup/cpu.stat
```

Memory

The limit counts the working set: anonymous memory, page cache that cannot be dropped cheaply and kernel memory attributed to the container. Page cache that can be reclaimed does not cause a kill by itself. `kubectl top` shows the working set, which is what the kubelet uses for eviction decisions. Languages with managed heaps need explicit limits that respect the container, such as a Java heap percentage or Go memory limit.

The OOM killer

When a cgroup hits its limit and reclaim fails, the kernel kills a process in it. Kubernetes sets oom_score_adj so that BestEffort processes are chosen first on node pressure and Guaranteed last. A kill shows as exit code 137 and reason OOMKilled. If the main process is killed, the container restarts with backoff. A killed child process may not be visible at all.

Tuning in practice

Watch usage and throttling over a week that includes peaks. Increase memory limits to cover the peak with margin and fix leaks rather than raising them endlessly. Choose CPU limits deliberately: leave them off for latency critical services or set them well above the request. Tune the runtime too, for example worker counts, to match the CPU you actually get.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: CPU and memory behaviour: throttling and out-of-memory kills

Part	What it does	What goes wrong
CPU request	Relative share.	Too low.
CPU limit	Quota per period.	Throttled.
cgroup	Applies both.	Misread.
Throttling	Paused time.	Latency spikes.
OOM kill	Exit 137.	Restart loop.

Flow: CPU request to cgroup / CPU limit to cgroup / cgroup to Throttling / cgroup to OOM kill

Incident: Latency spikes with low average CPU

> **Incident:** *A service with a CPU limit of one core had periodic latency spikes while dashboards showed 30 percent CPU use.*

```
$ kubectl exec api-1 -- cat /sys/fs/cgroup/cpu.stat
usage_usec 91240000
nr_periods 18200
nr_throttled 6100
throttled_usec 41200000
(this is a simulated illustration, not a real incident)
```

A third of all periods were throttled. The application ran eight worker threads that burned the quota early in each period and then waited. Raising the limit and reducing the worker count to match removed the spikes. Average CPU hides throttling, so look at the throttled share directly.

Common mistakes

- Reading average CPU as proof there is no throttling
- Setting tiny CPU limits on multi-threaded services
- Raising memory limits to hide leaks
- Ignoring runtime tuning inside the container
- Treating page cache as application memory

Check yourself

What does a CPU limit become in the kernel?

Answer: A CFS quota per period

Why can latency spike at low average CPU?

Answer: The container is throttled in bursts

What does exit code 137 with OOMKilled mean?

Answer: The container was killed for exceeding memory

Which metric ratio shows throttling?

Answer: Throttled periods to total periods

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. LimitRange, ResourceQuota and priority classes

12 minutes. Use LimitRange for defaults and bounds, ResourceQuota to cap a namespace, and PriorityClass to control who wins when capacity is short.

Without guardrails, one team can request all the cluster. Namespace level objects set defaults and ceilings, and priorities say which workloads matter when there is not enough for everyone.

What you will learn

- LimitRange defaults, minimums and maximums
- ResourceQuota for compute, storage and object counts
- What happens when a quota blocks a pod
- PriorityClass and preemption
- A model for sharing a cluster

LimitRange

A LimitRange in a namespace supplies default requests and limits for containers that omit them and enforces minimum and maximum values per container or pod. Defaults mean no pod is BestEffort by accident, and maximums stop a single container from asking for an entire node.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: shop
spec:
  limits:
    - type: Container
      default:
        memory: 512Mi
      defaultRequest:
        cpu: 100m
        memory: 256Mi
      max:
        memory: 2Gi
```

ResourceQuota

A ResourceQuota caps the total of requests, limits, storage and object counts in a namespace, for example total CPU requests, number of pods or persistent volume claims. When a quota covers compute resources, every pod must specify those resources or get them from a LimitRange, otherwise creation is rejected with a message naming the quota.

When quotas bite

A request that would exceed the quota is refused at admission with "exceeded quota". The status of the quota shows used and hard values. Quotas apply at creation, so a Deployment hitting a quota shows a ReplicaSet that cannot create pods, not a Pending pod. Look at ReplicaSet events.

```
kubectl describe resourcequota -n shop
kubectl describe rs -n shop | grep -i -A2 "quota"
```

Priority

A PriorityClass assigns an integer priority to pods. The scheduler queues higher priority pods first, and when no node fits can preempt lower priority pods. Use a few classes: system critical for platform components, high for important services, default for most and low for batch that may be displaced. Setting everything to high removes the benefit.

MARGA by EknathaLabs marga.eknathalabs.com

Sharing a cluster

A common model is one namespace per team with a quota sized to the team share, defaults from a LimitRange, and priorities for production over batch. Review quota use against actual use regularly, and add a report so teams see their own consumption.

Diagram: LimitRange, ResourceQuota and priority classes

Part	What it does	What goes wrong
LimitRange	Defaults and bounds.	Absent.
ResourceQuota	Namespace ceiling.	Too small.
Admission	Rejects over quota.	Unclear errors.
PriorityClass	Order and preemption.	Everything high.

Flow: LimitRange to ResourceQuota / ResourceQuota to Admission / Admission to PriorityClass

Incident: Pods that were never created

Incident: *A deployment showed zero pods and no Pending ones after a scale up.*

```
$ kubectl get deploy web
NAME   READY   UP-TO-DATE   AVAILABLE
web    2/6     2            2
$ kubectl describe rs web-6b5c | grep -i quota
Warning  FailedCreate  ... forbidden: exceeded quota: team-quota, requested: requests.cpu=500m, us \
    ed: requests.cpu=3800m, limited: requests.cpu=4
(this is a simulated illustration, not a real incident)
```

The namespace CPU request quota was nearly used, so new pods were refused when the ReplicaSet tried to create them. There were no Pending pods because pods did not exist. Raising the quota for a justified need, or reducing requests on over-provisioned pods, fixed it. Check ReplicaSet events when pods are missing rather than Pending.

Common mistakes

- Looking for Pending pods when the quota refuses creation
- Setting a quota without defaults
- Making every workload high priority
- Setting LimitRange maximums that block legitimate jobs
- Never reviewing quota against usage

Check yourself

What does a LimitRange default provide?

Answer: Requests and limits for containers that omit them

What does a ResourceQuota limit?

Answer: Totals per namespace such as CPU requests and object counts

Where do quota failures show up for a Deployment?

Answer: In ReplicaSet events

What can a higher PriorityClass do?

Answer: Preempt lower priority pods when nothing fits

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Node allocatable, pressure and eviction

12 minutes. Learn how node capacity becomes allocatable, how the kubelet detects memory, disk and PID pressure and which pods it evicts.

A node never gives pods all of its capacity. Some is reserved for the system and some is a safety margin. When the margin is used up, the kubelet evicts pods to protect the node, and knowing its rules explains which pods go first.

What you will learn

- Capacity, allocatable and the reservations in between
- Eviction signals and thresholds
- Soft and hard eviction and the order of victims
- Disk pressure and image garbage collection
- Reading node conditions

Allocatable

Capacity is what the machine has. Allocatable is what is left for pods after `kube-reserved` for Kubernetes daemons, `system-reserved` for the operating system and the hard eviction threshold. The scheduler works with allocatable. Without reservations, pods can starve the kubelet and the node may fall over before eviction helps.

Signals and thresholds

The kubelet watches memory.available, nodefs.available, imagefs.available, nodefs.inodesFree and pid availability. Hard thresholds trigger immediate eviction, for example memory.available below 100Mi. Soft thresholds have a grace period and trigger graceful eviction. When a signal passes its threshold, the node reports a pressure condition such as MemoryPressure, and new BestEffort pods are not scheduled there.

Choosing victims

Under memory pressure the kubelet ranks pods by whether they exceed their requests, then by priority, then by how much they exceed. BestEffort and Burstable pods over request go first. Guaranteed pods and those below their request are protected. Evicted pods are marked Failed with reason Evicted, and a controller recreates them elsewhere.

Disk

Disk pressure comes from container logs, images, writable layers and emptyDir volumes. The kubelet garbage collects unused images and dead containers when disk use crosses a threshold, then evicts pods if that is not enough. Set ephemeral storage requests and limits, rotate logs and avoid writing large files in containers.

```
kubectl describe node worker-2 | sed -n '/Conditions/,/Addresses/p'
kubectl get pods -A --field-selector=status.phase=Failed | grep Evicted | head
df -h /var/lib/containerd /var/lib/kubelet
```

Tuning

Set reservations from measurement and leave them in place on every node pool. Alert on pressure conditions and eviction counts. Clean up Evicted pods, which stay until deleted. Prefer fixing the workload that exceeds its requests over raising thresholds.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Node allocatable, pressure and eviction

Part	What it does	What goes wrong
Capacity	Machine size.	Not reserved.
Allocatable	What pods can use.	Too generous.
Eviction signals	Memory, disk, PIDs.	Late.
Eviction	Chosen by QoS and usage.	Wrong pods hit.

Flow: Capacity to Allocatable / Allocatable to Eviction signals / Eviction signals to Eviction

Incident: A disk that filled with logs

Incident: *A node began evicting pods one after another overnight.*

```
$ kubectl describe node worker-2 | grep -E "DiskPressure|Evicted"
  DiskPressure     True    KubeletHasDiskPressure   kubelet has disk pressure
$ du -sh /var/log/pods | tail -1
38G     /var/log/pods
(this is a simulated illustration, not a real incident)
```

A chatty application wrote gigabytes of logs without rotation limits, so the node file system crossed its threshold. The kubelet cleaned images, then evicted pods, including healthy ones. Setting log rotation limits, an ephemeral storage limit on the workload and an alert on disk use prevented a repeat.

Common mistakes

- Running nodes without kube-reserved and system-reserved
- Ignoring disk and inode pressure
- Raising eviction thresholds instead of fixing the workload
- Leaving Evicted pods in place
- Not setting ephemeral storage limits

Check yourself

What is allocatable?

Answer: Capacity minus reservations and eviction threshold

Which pods does the kubelet protect most under memory pressure?

Answer: Guaranteed pods and those below their requests

What is a common cause of DiskPressure?

Answer: Container logs, images and writable layers

What do you do with Evicted pods?

Answer: Delete them after finding the cause

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Horizontal pod autoscaling

13 minutes. Configure the HorizontalPodAutoscaler for CPU, memory and custom metrics, understand its formula, stabilisation and behaviour settings, and avoid flapping.

The HorizontalPodAutoscaler changes the replica count of a workload in response to load. It is simple in concept and easy to misconfigure, because its decisions depend on requests, metrics and timing.

What you will learn

- The HPA control loop and its formula
- Why CPU utilisation depends on requests
- Metrics sources: resource, custom and external
- Stabilisation windows and scaling behaviour
- Common misconfigurations

The loop

Every fifteen seconds by default the HPA controller reads a metric for the target pods, compares it to the target value and computes the desired replicas as the current replicas multiplied by the ratio of current to target metric, rounded up. It then updates the scale subresource of the Deployment or StatefulSet. A tolerance of ten percent avoids small changes.

```
import math
def desired(current, metric, target):
    return math.ceil(current * metric / target)
print(desired(4, 90, 60))
print(desired(10, 30, 60))
```

Utilisation is relative to requests

For a CPU utilisation target such as 60 percent, utilisation means usage divided by the pod request. If a container has no CPU request, the HPA cannot compute it and reports unknown. A request set too low makes utilisation look high and causes over-scaling, and a request set too high hides load. Set requests first, then targets.

Metrics sources

Resource metrics come from the metrics-server, which must be installed. Custom metrics, such as requests per second per pod, come through an adapter like the Prometheus adapter, and external metrics describe things outside the cluster, such as queue length. KEDA builds on the HPA to scale from event sources, including down to zero. Scaling on a metric that reflects the real bottleneck works better than CPU for many services.

Behaviour

The `behavior` field controls speed. Scale up defaults to fast, doubling or adding four pods per period, while scale down waits for a five minute stabilisation window and removes pods more slowly. You can set policies, for example at most ten percent a minute down, to avoid flapping.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

Check it

```
kubectl get hpa web
kubectl describe hpa web
kubectl top pods -l app=web
```

The describe output shows current metrics, the conditions such as AbleToScale and ScalingActive and recent scaling events. A target of unknown usually means no requests or no metrics-server.

Diagram: Horizontal pod autoscaling

Part	What it does	What goes wrong
metrics-server	Resource metrics.	Missing.
HPA controller	Computes replicas.	No requests.
Scale subresource	Updates replicas.	Fights manual scaling.
Pods	Receive load.	Slow to start.

Flow: metrics-server to HPA controller / HPA controller to Scale subresource / Scale subresource to Pods

Incident: An autoscaler that showed unknown

Incident: *A new HPA never changed the replica count despite heavy load.*

```
$ kubectl get hpa web
NAME   REFERENCE        TARGETS         MINPODS   MAXPODS   REPLICAS
web    Deployment/web   <unknown>/60%   3         20        3
$ kubectl describe hpa web | grep -A2 Conditions
  ScalingActive  False  FailedGetResourceMetric  the HPA was unable to compute the replica count:  \
    failed to get cpu utilization: missing request for cpu
(this is a simulated illustration, not a real incident)
```

The container had no CPU request, so utilisation could not be computed and the HPA stayed idle. Adding a CPU request let it compute utilisation, and it scaled up within a minute. If TARGETS shows unknown, check requests and the metrics-server before touching the HPA.

Common mistakes

- No CPU request on the target
- Scaling on CPU when the bottleneck is elsewhere
- Setting minReplicas to one for a critical service
- Editing replicas by hand while an HPA manages them
- Forgetting that new pods take time to become ready

Check yourself

What does the HPA formula use?

Answer: Current replicas times current metric over target metric, rounded up

Why does CPU utilisation need requests?

Answer: Utilisation is usage relative to the request

What gives resource metrics to the HPA?

Answer: The metrics-server

What does the scale down stabilisation window do?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Delays removal of pods to avoid flapping

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Vertical scaling and right-sizing

12 minutes. Use the Vertical Pod Autoscaler, in-place resize and usage data to right-size requests, and know why resizing and horizontal scaling interact.

Most clusters waste capacity because requests were set once and never revisited. Right-sizing recovers it, and the Vertical Pod Autoscaler helps by recommending or applying better numbers.

What you will learn

- Why requests drift from reality
- VPA modes: Off, Initial, Recreate and Auto
- In-place pod resize
- Using recommendations safely
- Combining with the HPA

Drift

Teams copy requests from examples or set them generously after an incident. Over time many pods request several times what they use, so nodes look full while actual utilisation is low. A cluster at 30 percent real utilisation can need twice the machines it should. The reverse also happens: requests too low give noisy neighbours and OOM kills.

The Vertical Pod Autoscaler

VPA is an add-on that watches usage history and computes recommended requests for each container. Its updater and admission controller can apply them. In mode Off it only reports recommendations. Initial sets requests when pods are created. Recreate evicts and recreates pods with new values. Auto chooses an update method, and newer versions can use in-place resize where supported.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: api
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  updatePolicy:
    updateMode: "Off"
```

In-place resize

Recent Kubernetes versions allow changing container CPU and memory requests and limits on a running pod without recreating it, subject to node capacity and the resize policy of each container. This reduces disruption for stateful and slow starting workloads. Check the feature status for your version.

Using recommendations

Start with mode Off and read the recommendation, which includes lower bound, target and upper bound. Compare to real peaks, apply to a canary and watch. Treat the output as advice, especially for workloads with rare spikes the history did not include. Avoid Auto on single replica services unless an interruption is acceptable.

```
kubectl describe vpa api | sed -n '/Recommendation/,$p'
```

With the HPA

Do not let both the HPA and VPA act on the same resource metric, such as CPU, for the same workload, because each reacts to the other. A common arrangement is HPA on CPU or a custom metric, and VPA only recommending for memory, or VPA in Off mode while you set values by hand.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: Vertical scaling and right-sizing

Part	What it does	What goes wrong
Usage history	Observed demand.	Short window.
Recommender	Computes targets.	Missed peaks.
Apply	Recreate or resize.	Disruption.
Capacity saved	Fewer nodes.	Under-sized.

Flow: Usage history to Recommender / Recommender to Apply / Apply to Capacity saved

Incident: A recommendation that missed the monthly peak

Incident: *After accepting VPA targets, a service restarted on the first day of the month.*

```
$ kubectl describe vpa api | sed -n '/Recommendation/,$p'
  Recommendation:
    Container Recommendations:
      Container Name:  api
      Target:
        Cpu:     120m
        Memory:  300Mi
$ kubectl describe pod api-9d | grep -A2 "Last State"
    Last State:  Terminated  Reason: OOMKilled
(this is a simulated illustration, not a real incident)
```

The usage history covered two weeks, so the end of month report job was never seen and the recommended memory was too small for it. Adding headroom, a longer window and a separate workload for the monthly job fixed it. Recommendations only know the past they were shown, so apply judgement for rare peaks.

Common mistakes

- Using Auto mode on single replica critical services
- Letting HPA and VPA fight over CPU
- Trusting a short history
- Setting requests once and never reviewing
- Right-sizing without a rollback plan

Check yourself

What does VPA mode Off do?

Answer: Only produces recommendations

Why do requests drift?

Answer: They are set once and rarely revisited

What does in-place resize allow?

Answer: Changing resources on a running pod without recreating it

How should HPA and VPA be combined?

Answer: Avoid both acting on the same metric for one workload

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Cluster autoscaling, Karpenter and KEDA

12 minutes. See how the Cluster Autoscaler and Karpenter add and remove nodes in response to pending pods, and how KEDA scales workloads from events, including to zero.

Pod autoscaling is only useful if there are nodes to place the new pods. Node autoscaling closes that gap, and event driven autoscaling lets workloads follow queues rather than CPU.

What you will learn

- The Cluster Autoscaler and what it reacts to
- How scale down decisions are made and blocked
- Karpenter and just-in-time node provisioning
- KEDA and scaling to zero
- Cost and safety guardrails

Cluster Autoscaler

The Cluster Autoscaler watches for pods that are Pending because no node has room. It picks a node group whose template would fit them, simulates the placement and increases that group size. It also finds nodes that have been underused for a period, checks that their pods can run elsewhere, and removes them. It works with node groups defined in the cloud provider, so node shapes are fixed by those groups.

Blocked scale down

Scale down is blocked by pods that cannot be moved: pods with local storage, pods not backed by a controller, restrictive PodDisruptionBudgets, pods with the annotation `cluster-autoscaler.kubernetes.io/safe-to-evict` set to false, and system pods in some configurations. A cluster that never shrinks often has one such pod on every node.

Karpenter

Karpenter takes a different approach. It watches Pending pods directly, evaluates their combined requirements, and launches nodes of suitable types and sizes through the cloud API, without predefined node groups. It also consolidates by replacing or removing nodes to reduce cost, and can handle spot interruptions. You define NodePools with limits and constraints such as instance families, zones and capacity types.

KEDA

KEDA adds scalers for event sources, such as message queues, streams, databases and schedules, and exposes them as external metrics to the HPA. Its main addition is scaling to zero and back when events arrive. A queue worker with an empty queue then costs nothing, and starts when messages appear.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: worker
spec:
  scaleTargetRef:
    name: worker
  minReplicaCount: 0
  maxReplicaCount: 30
  triggers:
    - type: rabbitmq
      metadata:
        queueName: jobs
        value: "20"
```

MARGA by EknathaLabs
marga.eknathalabs.com

Guardrails

Set maximum node counts or limits so a runaway workload cannot create an unbounded bill. Spread across zones. Keep PodDisruptionBudgets for important workloads so consolidation is safe. Make sure daemons and system add-ons have enough requests. Test scale up latency, since node start time adds to the time before new pods run, and keep a small buffer of spare capacity if that delay is unacceptable.

Diagram: Cluster autoscaling, Karpenter and KEDA

Part	What it does	What goes wrong
Pending pods	Signal for scale up.	Wrong requests.
Autoscaler or Karpenter	Adds nodes.	Quota.
New node	Joins and becomes Ready.	Slow start.
Consolidation	Removes idle nodes.	Blocked pods.

Flow: Pending pods to Autoscaler or Karpenter / Autoscaler or Karpenter to New node / New node to Consolidation

Incident: The cluster that never scaled down

Incident: *A cluster stayed at its peak node count for weeks after a one-day load test.*

```
$ kubectl -n kube-system logs deploy/cluster-autoscaler | tail -2
I1114 06:10:01 scale_down.go:  node ip-10-0-3-21 cannot be removed: pod kube-system/metrics-agent  \
    is not replicated
$ kubectl get pods -A -o wide | grep ip-10-0-3-21 | wc -l
11
(this is a simulated illustration, not a real incident)
```

A single unmanaged pod with local storage sat on the node and blocked its removal, and similar pods blocked others. Moving the agent to a DaemonSet, adding safe-to-evict annotations where valid and reviewing PodDisruptionBudgets let nodes drain and the cluster shrink. If a cluster never scales down, ask the autoscaler why in its log.

Common mistakes

- Leaving unmanaged or non-evictable pods on every node
- No maximum node limit
- Autoscaling nodes without requests on pods
- Ignoring node start time in latency targets
- Using consolidation without PodDisruptionBudgets

Check yourself

What triggers Cluster Autoscaler scale up?

Answer: Pods that are Pending because no node has room

What blocks scale down?

Answer: Pods that cannot be moved, such as unmanaged pods

How does Karpenter differ?

Answer: It provisions suitable nodes directly from pending pod requirements

What does KEDA add?

Answer: Event driven scaling including to zero

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Disruption budgets, topology spread and capacity planning

13 minutes. Protect availability during voluntary disruption with PodDisruptionBudgets and topology spread, and plan capacity with headroom for failures and growth.

Autoscaling and upgrades both move pods around. Disruption budgets and spread constraints say how much movement a workload can take, and capacity planning makes sure there is room to move into.

What you will learn

- PodDisruptionBudgets: minAvailable and maxUnavailable
- Voluntary versus involuntary disruption
- Topology spread constraints and anti-affinity
- Planning N plus one capacity
- A review checklist

Disruptions

Voluntary disruptions are actions by operators and tools: drains, upgrades, consolidation and eviction API calls. Involuntary ones are hardware failures, kernel panics and node loss. PodDisruptionBudgets apply to voluntary evictions made through the eviction API, so they do not stop a node crashing, but they do slow a drain.

PodDisruptionBudgets

A PDB selects pods and declares `minAvailable` or `maxUnavailable`, as a number or a percentage. The eviction API refuses an eviction that would break it. Choose values that match the workload: a three replica service might allow one unavailable. A budget of zero allowed disruptions blocks all drains, which is the cause of many stuck upgrades.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: web
```

Spreading

Topology spread constraints tell the scheduler to keep replicas evenly spread over zones or nodes, within a maximum skew. They are clearer than pod anti-affinity for most uses. Combine them with several replicas so that the loss of a zone or node leaves enough capacity.

```
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: web
```

Capacity planning

Plan so that the cluster survives the loss of the largest failure domain you care about. With three zones, that means the remaining two can carry the load, so run each zone at no more than about two thirds of its capacity. Add headroom for the autoscaler delay and for upgrades, which need spare room to drain into. Review growth trends monthly.

```
zones = 3
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
load = 100.0
capacity_per_zone = load / (zones - 1) * 1.15
print("capacity per zone:", round(capacity_per_zone, 1))
print("total:", round(capacity_per_zone * zones, 1))
```

Checklist

For each production workload: at least two or three replicas, a PDB that allows one disruption, a spread constraint over zones, requests that reflect reality, a readiness probe and a graceful shutdown. For the cluster: node headroom, autoscaler limits, and a tested drain of one node and one zone.

Diagram: Disruption budgets, topology spread and capacity planning

Part	What it does	What goes wrong
PodDisruptionBudget	Limits evictions.	Zero allowed.
Topology spread	Even placement.	One zone.
Capacity headroom	Room to move.	Full nodes.
Drain test	Proves it.	Never run.

Flow: PodDisruptionBudget to Topology spread / Topology spread to Capacity headroom / Capacity headroom to Drain test

Incident: A zone loss that took the service down

Incident: *A single zone failed and a service with six replicas went fully unavailable.*

```
$ kubectl get pods -l app=web -o wide --no-headers | awk '{print $7}' | sort | uniq -c
      6 node-a1
$ kubectl get nodes -L topology.kubernetes.io/zone | grep node-a1
node-a1   Ready   zone-a
(this is a simulated illustration, not a real incident)
```

All six replicas had been scheduled to one node in one zone because no spread constraint was set and that node had the most free resources. When the zone failed, nothing was left. A zone spread constraint, a PDB and a capacity plan with headroom in the other zones would have kept the service up. Replica count is not availability unless the replicas are spread.

Common mistakes

- Several replicas on one node or zone
- A PDB with no allowed disruptions
- Running every zone at high utilisation
- No test of a drain or a zone loss
- Confusing voluntary and involuntary disruption

Check yourself

What do PodDisruptionBudgets limit?

Answer: Voluntary evictions through the eviction API

What does maxSkew control?

Answer: How uneven the spread of pods can be across a topology

How much utilisation should each of three zones run at to survive losing one?

Answer: About two thirds or less

Why test a drain?

Answer: It reveals blocked PDBs and capacity gaps before an upgrade

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Resource management and autoscaling

Requests and limits, QoS classes, throttling checks, quota debugging and autoscaler reference.

Requests and limits

Setting	Effect
CPU request	Scheduling and share of CPU
CPU limit	Throttling above the quota
Memory request	Scheduling and eviction order
Memory limit	OOM kill above the limit

QoS classes

Class	Rule
Guaranteed	Requests equal limits for CPU and memory in every container
Burstable	At least one request or limit set
BestEffort	None set, evicted first

Checks

Job	Command	
Usage	`kubectl top pods`	
Why restarted	`` `kubectl describe pod NAME ``	grep -A4 "Last State"`
Throttling	`cat /sys/fs/cgroup/cpu.stat` in the container	
Quota use	`kubectl describe quota -n NS`	
HPA	`kubectl describe hpa NAME`	

Autoscaling

Tool	Scales
HPA	Replicas from metrics
VPA	Requests from history
Cluster Autoscaler or Karpenter	Nodes for pending pods
KEDA	Replicas from events, including to zero

MARGA by EknathaLabs marga.eknathalabs.com